

OKR Component Metric

1. Component Metrics [↗](#)

Component metrics are specialized tools or functionalities in Qollabi designed to gather data points, help users analyze and track these data points, manage goals, and provide insights. This is with the goal to help teams gather data points, track progress, make informed decisions, and collaborate more effectively.

A component metric is a combination of a set of **properties**. There can be more than one component metric in an environment, and they can be used in different parts of the platform.

For example, at the moment, 3 component metrics are identified:

- **OKR (in scope)**
- Survey (later)
- Guidance (later)

Each of these component metrics has a different group of properties that have been turned on, from a larger set of available properties.

Because there are multiple properties, and those can be configured differently, component metrics can look different in different environments.

Component Metric Properties [↗](#)

Each **Component Metric** comes with a **default set of properties** that make sense for its functionality. These properties are pre-selected and designed to target our customers' use cases (e.g., **Progress, Due Date, Attachments, Name, Realized**).

This is the full list of **properties** that can be used to create component metrics. This list will expand or adapt over time.

Property	Type
Name	Text
Description	Multilinetext
Realized	Input depends on the measurement unit selected
Target	Input depends on the measurement unit selected
Measure Unit	
Currency	Currency
Number	Number
Checkbox	Boolean
Percent	Percent

Progress	Progress bar
Traffic light / Status	Single select picklist with status and icon
Hierarchy	Objective, activity, or subactivity
Responsible	Multi-select picklist (User and contact relationship)
History	Log to track all entries
Timeframe	Date picker (date range)
Frequency (milestone)	
Attachment	
Due Date	Date picker
Trend chart	Trend chart graph
Muted/Unmuted	
Shared/Private	
Comments	
Link	URL

Customization [🔗](#)

Some component metrics allow customers to turn certain properties on or off depending on their specific needs or environment.

For example, for an OKR component metric, one customer might not want to have attachments.

Standard Component Metrics [🔗](#)

The **Standard Component Metrics** are pre-defined features integrated into the platform that we have identified. In the future, we might open the door for customers to create their own.

For now, we will only describe the **OKR** component metric as it will be the first one that we will develop and will be available on all environments.

1. OKR (Objectives and Key Results) Component Metric [🔗](#)

- **Purpose:** OKRs help align team or organizational goals with measurable outcomes. They are key to tracking performance against set objectives and key results.
- **Properties:**
 - Name
 - Description

- Realized
- Target
- Measure unit of the realized and target
 - Currency
 - Number
 - Percent: A field specifically designed for percentage values (e.g., 50%).
 - Checkbox (complete/not complete)
 - Picklist (single-choice)
 - Picklist (multiple-choice)
- Traffic light
 - A threshold can be defined. For example, *when the realized is between x and y, the traffic light color should be w.*
- Progress bar
 - A threshold can be defined. For example, *when the realized is between x and y, the progress bar should be w.*
- Trend chart
- Responsible
 - Relationship with users
 - Relationship with contacts
- Timeframe
- Frequency/Milestone
 - Yearly
 - Quarterly
 - Monthly
 - Weekly
 - None
 - Custom
- Attachment
- Due date
- Muted/unmuted → product team still defining
- Archived → product team still defining
- Shared/private → product team still defining
- Comments
 - Relationship with users
 - Relationship with contacts
- Hierarchy
 - Objective
 - Activity
 - Subactivity
- Tag
- History

A. Name

Text

Name is always required; this can't be modified.

B. Description

MultilineText

Later, we'll include a rich text editor.

When an OKR has a description, an icon will appear next to the OKR name. When the user hovers over it, he can read the description.

Can be optional or required

C. Realized

Can be optional or required

The value in the realized field can be updated manually by the users (considering they have permission to do so), by uploading data into Qollabi AI via an Excel or CSV file, or through a data integration.

This field is connected to:

- **Measure unit:** The format of the realized field will change depending on the **Measure unit** selected.
- **Target:** Realized is measured against the target.
- **Progress:** How much, in %, is the realized out of the target.
- **Traffic light:** How is the realized doing against the target in colored statuses.
- **Trend chart:** Indicates every entry of the realized in a trend-line graph.
- **Responsible:** To identify who needs to work on the OKR.
- **Due date:** To identify when the realized value should reach the target.
- **Milestone:** Represents the amount of times the realized value should reach the target.
- **History:** Log every time the realized changes, either manually or via a data integration.

The realized can be empty or it can have a value.

When creating an OKR, the user can choose to define a default realized or he can leave it `empty`.

When left empty, the realized field will be displayed with a `-`

Important: today in Qollabi, when the realized is empty, it displays a 0. This is a wrong behavior, because 0 can mean an actual value.



As a user,

I want the realized field in a new OKR to default to an empty state (displayed as a dash `-`) rather than 0, **so that** I don't mistakenly assume that 0 represents an actual measured value and can accurately track when no progress has been recorded.

Acceptance Criteria [🔗](#)

Visual Representation [🔗](#)

- When creating a new OKR, the realized field displays an empty state (represented by a dash `-`) instead of a numeric 0.

- The empty state (dash -) follows the approved design guidelines and is visually distinct from any valid numeric entry.

User Interaction [🔗](#)

- Users can manually enter a numeric value (including 0) into the realized field, and the system treats this as an intentional update.
- When editing an existing OKR, if no value is provided, the realized field remains displayed as a - until the user inputs a value.

System Behavior & Integration [🔗](#)

- The system differentiates between an empty realized field (-) and a numeric value of 0 in all displays and calculations.
- Any automated processes or integrations referencing the realized field interpret the empty state as “no value recorded” rather than a 0.
- If the user clears the realized field during an update, it reverts to displaying an empty state (-).

PP Focus Partner Plan 2025 ▾		Realized ▾	Target ▾
Revenue Goal		560.000 €	- €
> Pipeline New Business 3 FY25		- €	2.000.000 €

D. Target

Can be optional or required

The value in the target field can be updated manually by the users, by uploading data into Qollabi AI via an Excel or CSV file, or through a data integration.

This field is connected to:

- **Measure unit:** The format of the target field will change depending on the **Measure unit** selected.
 - You need to have a measure unit selected in order to create an OKR.
 - Usually, we might have a predefined unit already to facilitate the easy and quick creation for users. For example, we might say that by default, the measure unit selected is a checkbox (complete/not complete).
- **Realized:** Realized is measured against the target.
- **Progress:** How much, in %, is the realized out of the target.
- **Traffic light:** How is the realized doing against the target in colored statuses.
- **Trend chart:** Indicates every entry of the target in a trend-line graph.
- **Responsible:** To identify who needs to work on the OKR.
- **Due date:** To identify when the realized value should reach the target.
- **Milestone:** Represents the amount of times the realized value should reach the target.
- **History:** Log every time the target changes, either manually or via a data integration.

The target can be empty or it can have a value.

Similar to the realized, leaving a target empty will be indicated with a **-**, instead of a 0.

- As a user,**

I want the target field in an OKR to default to an empty state (displayed as a dash **-**) instead of 0,

so that I can clearly distinguish between a missing target and a target that is intentionally set to 0.

Acceptance Criteria

Visual Representation

- When a new OKR is created and the user hasn't defined a target, the target field displays an empty state (represented by a dash **-**) instead of a numeric 0.
- The empty state (**-**) is styled consistently with the approved design guidelines and is easily distinguishable from valid numeric entries.

User Interaction

- Users can manually enter a numeric value—including 0—to indicate an actual target.
- When editing an existing OKR, if no value is provided, the field remains displayed as a dash **-** until the user enters a new value.

System Behavior & Integration

- The system clearly differentiates between an empty target (**-**) and a numeric target of 0 in all relevant displays.
- Any integration or automated process referencing the target field recognizes the empty state as “no target provided” rather than as a zero value.
- If the user clears the target field during an update, it reverts to displaying an empty state (**-**).

PP Focus Partner Plan 2025	Realized	Target
Revenue Goal	560.000 €	- €

- The target should be optional when creating an OKR:

- As an internal user,**
- I want** to be able to create a new OKR without being forced to define a target,
- so that** I can define the targets later for my partners (if applicable).

Acceptance Criteria

Visual Representation

- The **Milestone Target** and **Overall Target** fields display an **empty state**.
- If the user leaves the **target field empty**, it is stored and displayed as a **dash (-)** instead of a numeric value.
- The field remains visually distinguishable from an actual numeric target.

User Interaction

- The milestone target field is **optional** when creating an OKR.

- Users can:
 - Leave the field empty.
 - Enter **any numeric value**, including **0**, if they choose.
- If left empty, the field **displays a dash (-) upon saving** instead of a 0.

System Behavior & Integration [🔗](#)

- The system **correctly differentiates** between:
 - An **empty target** (displayed as "-") and
 - A **target explicitly set to 0**.
- When the OKR is assigned to records, the target remains as **"-" (empty state)** if it was not defined.
- Any **integrations** or **automated processes** referencing the target field interpret the empty state as “no target provided.”
- If the **target field is later updated**, the change is **immediately reflected** across all relevant views.

E. Measure unit

Defines the format and input field for the `realized` and `target` properties.

Options:

- **Currency:** Follow the currency settings of the environment. The currency sign should be at the front, inside the input field.
 - Anticipate how it's stored for ALE use case...
- **Number:** Follow the number and decimal settings of the environment. The number sign should be at the back, inside the input field.
- **Percent:** A field specifically designed for percentage values (e.g., 50%). The number sign should be at the back, inside the input field.
- **Checkbox** (complete/not complete)
- **Picklist (single-choice)** → not in scope now
- **Picklist (multiple-choice)** → not in scope now

A measure unit is required; this can't be modified.

However, we can define, per environment, which should be the default measure unit selected when creating an OKR.

For example, we can define that by default, the measure unit preselected is a checkbox (complete/not complete) – because this might be the most used for opportunity management.

We should consider being able to change this per environment, as per the customer's request. In a later version, we can think about adding it to their environment settings so that they can define it themselves.

The measure unit could be defined by the users manually (considering they have the permissions to do so), via a data upload (Excel/CSV) when creating new OKRs, or data integration that creates a new OKR.

F. Progress

The progress is used to define how much, in %, is the realized out of the target. This helps users understand visually how far they are from reaching their target.

The progress consists of:

- A progress bar with color codes
- A % in front of the progress bar
- An “Ideal result”

Progress can be configured in three ways:

1. **System**

System progress follow the rules that we have defined in Qollabi AI. The system calculates the progress based on the realized vs. target in the same way as in Qollabi 1.0. We need to do an investigation what the exact rules are and behavior.

2. **Custom threshold**

The user can define how the progress bar should behave.

They can define the percentage distribution, based on the measure unit selected. They need to distribute the % so that it is equal to 100%.

For example:

0% is grey

between 1% and 10% is red

between 20% and 70% is orange

between 71 and 99% is green

100% is complete

3. **Off**

Users need to be able to not have the progress bar displayed for some OKRs. In some use cases it doesn't make sense. For example, when they're measuring KPIs without a target.

G. Traffic light

Can be optional or required

The traffic light indicates how the `realized` is doing against the `target` using a status and a colorful indicator.

Traffic lights can be configured in different ways:

1. **System**

System traffic lights follow the rules that we have defined in Qollabi AI. The system calculates the traffic light based on the progress of realized vs. target in the same way as in Qollabi 1.0. We need to do an investigation what the exact rules are and behavior.

- The traffic light colors are:
 - Grey → Not defined
 - Red → At risk
 - Orange → Behind
 - Green → On track
 - Dark green with complete checkmark icon → Complete

Dark green (complete): is an OKR that has reached 100% completion

UX:

Users see lights change dynamically as realized or target values change.

Users can override the color manually.

- If the light was overridden manually, it stays manual, even if the underlying data changes.
- Users must explicitly rest to automatic if they want the system to take over again.

2. Custom Thresholds (Advanced)

A threshold can be defined by the user. For example, *when the realized is between x and y, the traffic light color should be w*.

A threshold needs to be indicated for every color.

UX:

The UI allows defining percentage brackets (e.g., “Red=0-40%”). Traffic lights update automatically based on this custom threshold.

Users can override the color manually.

- If the light was overridden manually, it stays manual, even if the underlying data changes.
- Users must explicitly rest to automatic if they want the system to take over again.

3. Manual

The traffic lights will only update when a user manually updates them, or if the color is updated via an upload or integration.

4. Off

Users might not want to use the traffic lights for specific OKRs, so they need to be able to disable them.

G. Trend chart

A trend chart is a graph that helps visualize how a realized is behaving over time, whether it's moving towards a target, becoming more or less variable, or displaying other patterns.

This chart is updated automatically by the system every time there is an entry – users can't edit the chart themselves. It is mainly beneficial whenever the progress bar is not activated for an OKR because it doesn't have a target. Then, they can have a quick preview of the trend for this OKR.

It cannot be marked as optional or required.

H. Responsible

The responsible field is used to identify who needs to work on an OKR.

This field should be connected to `users` and `contacts`.

It is a picklist where one or more people can be selected.

You can search by name or email. We can start with name, if email adds extra complexity.

It includes every user who has access to that specific tab.

Later, there will be a connection between `responsible` and smart updates, reports, notifications, lists, etc., so that users can have an aggregated view of everything they're responsible for across Qollabi, or get notified via email or in-app.

Can be optional or required.

I. Timeframe

OKRs can have a start and end date. Contrary to Qollabi 1.0, this start and end that are optional.

Based on them, the timeframe picker on the record page will be able to display the OKRs that apply. If there are OKRs with no start and end dates, they will always appear.

The start and end date affect the calculation of the progress (check how it works in Qollabi 1.0).

J. Frequency/Milestone

The user can choose which frequency or milestone is applicable for an OKR. The options are:

- Yearly
- Quarterly
- Monthly
- Weekly
- None
- Custom (not prio)

Whenever an OKR reaches the end date of a frequency (e.g., the last day of a week), then it will reset the target to the default and the realized back to empty (-) for the next frequency ahead.

When defining a frequency, there needs to be a start date and ideally, an end date.

Can there be a frequency without an end date?

Each frequency will sum up with each other to amount to a total. If there is an OKR with a quarterly target of 10 for a period of 4 quarters with an end date, then it has a total amount of 40 for the whole period. The same should apply for the realized.

However, the percentage is different. Today in Qollabi 1.0 with wrong behavior:

- e.g. 30% conversion rate → this is not something that should be reached at the end of the year, but it should also be already the case for January
- Percentages are added in absolute numbers.
 - e.g. I work with monthly milestones. The target is 30% in every month. In the accumulation, it will provide me the target: 360% instead of 30%
 - e.g. in measure this behavior is x10 with all the different partners that have the same behavior (a target of 1042 % does not make sense)

Instead, maybe it should be summed up as the average.

K. Attachment

Users can upload attachments to OKRs. Whenever an attachment is uploaded for an OKR, an icon will be visible and will indicate it.

Add an attachment to an OKR

On the record page, navigate to the objective you want to attach a file to. Click on the **attachment icon** to open the OKR details slider. Within that slider, you can manage files associated with the OKR.

Vision that we don't want to develop in Q4 yet

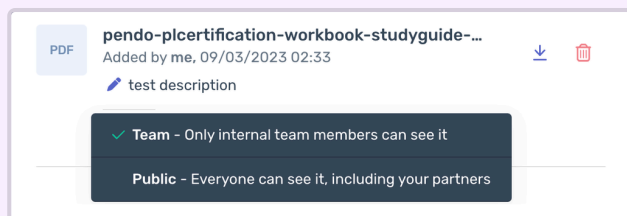
Within that slider, you should see various sections related to the objective, such as name, description, start-end date, milestones, responsible type, tags, key results, and permissions.

Users can add an attachment to an objective, activity, subactivity, custom objective, custom activity, custom subactivity, and quick action.

User permissions

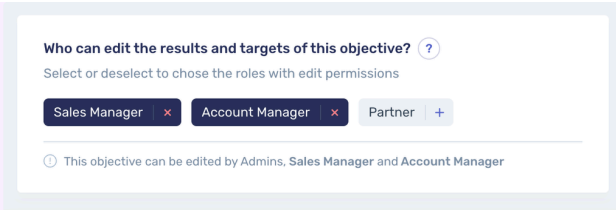
I) All users who can see the OKR can also see the attachments uploaded in it; it should follow the same behavior as the attachments slider we have today.

- This heavily depends on whether the attachment is marked as **team** or **public**.



Example of UI

II) If the user can see the OKR and has edit permission on that OKR, he can add an attachment to it.



Example of UI

II) If an OKR is muted, the user won't be able to add an attachment to it.

How to add the attachment [🔗](#)

You can drag and drop your file directly from your computer into this section. Alternatively, there might be an option to click within the 'Attachments' section to open a file dialog box on your computer, allowing you to select the file you want to attach.

After dragging and dropping the file, the system will initiate the upload process.

Once the upload is complete, you should see a confirmation message indicating that the attachment was successfully added. The file is now associated with the OKR.

When there's a new attachment uploaded to an OKR, there must be an indication with the attachment icon.

For now, the information for each OKR is limited to what you see on the line itself, plus on the tooltips if you hover. The idea is to create a modal or slider where the user has access to all information available for each OKR.

L. Due date

The due date is used to identify when the realized value should reach the target or when a task should be completed.

This is a calendar picker – users can select from any period of time.

 <https://www.figma.com/design/352ZV90ZFO01oTdh9sGDX/Qollabi.ai--product-wip-?node-id=514-46104> [Connect your Figma](#)

a account

Section 1	
Open questions:	
How do we deal with the decimals for realized and target?	Ability to change the name Realized/Target per plan as sometimes for one plan it's correct but the next plan on the same page it should be "forecast/realized" or something else
Do we round up/down the progress?	
How do we deal with (-) negative OKRs?	All hierarchies of the OKR should have the possibility to use traffic light and progress bar combo, there's no need to limit the activities & subactivities
	How do they create custom OKRs? Can they? Do they have default properties? Can they define default properties per environment or per plan?
	How do we deal with the request of not allowing their users to create custom OKRs within some specific plans or under some specific objectives?
	How should the currency function? considering the multi-currency use cases
How does archiving differ from muting, and do we still need muting?	
How do we display the OKRs with and without milestones?	
Can we reorder the columns?	
Can we change the size of the columns?	

Creating OKRs inline

As a user,

I want a simplified and efficient way to create OKRs,

So that I can quickly set objectives without navigating through multiple side drawers and fields, saving time and enhancing my productivity.

Acceptance criteria [🔗](#)

Where can I create OKRs inline [🔗](#)

- Record page → custom OKRs

Creating an OKR inline [🔗](#)

- Clicking on any of the 'add an OKR' buttons will display an inline input field
- A placeholder appears inside
- I can write text and press Enter on my keyboard to save it
- If I press anywhere outside the input bar, the OKR isn't created
- If I press ESC on my keyboard, the input is disregarded, and nothing is created
- I can do it for objectives, activities, subactivities, custom objectives, custom activities, custom subactivities

Default rules [🔗](#)

- Target unit: **Complete/Not complete**
- Start-end date: **This fiscal year**
 - The current year needs to follow the **fiscal year** selected from the Manage panel

- Milestone reporting frequency: **No milestones**

Edge case for all ALE environments [↗](#)

- Target unit: **Complete/Not complete**
- Start-end date: **This fiscal year**
 - The current year needs to follow the **fiscal year** selected from the Manage panel
- Milestone reporting frequency: **Yearly**

Selecting a unit type [↗](#)

- Inside the input field, there are options to select another unit type
- We display all of them: currency, number, percentage, complete/not complete
 - Complete/not complete has a new icon
- Before pressing ENTER on my keyboard and creating the OKR, I can switch between them freely

Selecting currency/number/percentage [↗](#)

- After I press ENTER and the OKR is created, I can fill the target.

		REALIZED / TARGET	PROGRESS	RESPONSIBLE	DUE DATE
	Objective A 289374823649183241678uwhtsdjksa...	2023 >	995 € / 1.000 € 99 %	 	BPe AQo 
	Objective Test		994 € / 1.000 € 99 %	 	
	Objective only traffic light		On track 		

Press ENTER to save or ESC to cancel







Save 

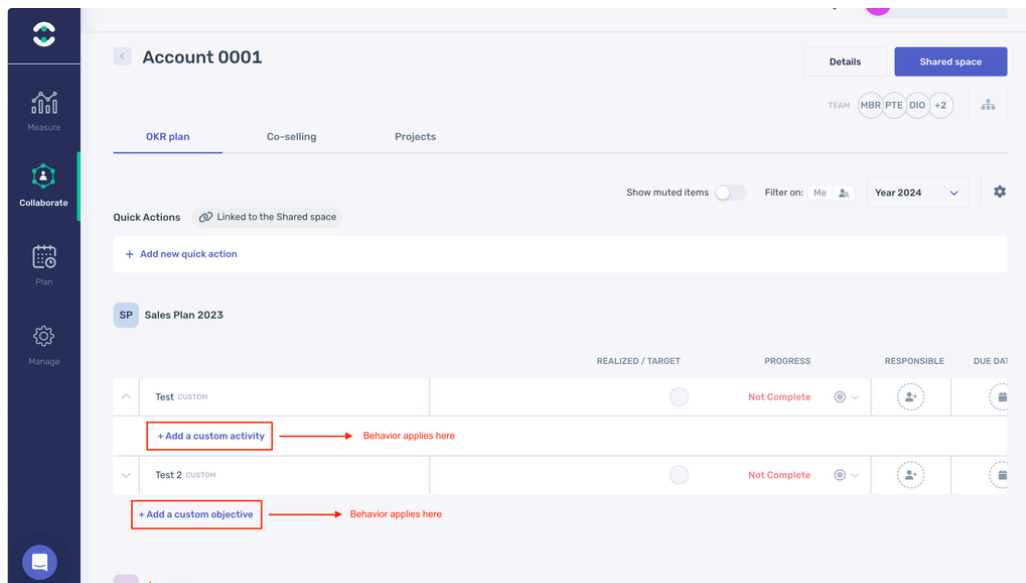


Opening the OKR slider [↗](#)

- In the inline component, there is an icon that facilitates users to open the OKR slider from there
- The name of the OKR is prefilled; if there is a name
- The user can fill in the rest of the details in the slider
- The user can click on Cancel and OKR won't be created
- The user can click on Save OKR and the OKR will be created

Edit OKR [↗](#)

- Once the OKR is saved, I can reorder it like the other ones in the plan
- Once the OKR is saved, I can edit it like the other ones in the plan
 - I can edit every aspect of the OKR



Creating & Assigning OKRs

You can assign OKRs to records using **OKR templates**, which come in different formats:

Types of OKR Templates [🔗](#)

1. Loose OKR Template

- Contains **just one OKR**, usually an **Objective** (not typically a sub-activity).
- These are standalone and not necessarily part of a structured plan.

2. Single Objective with Structure

- One **Objective** that includes **Activities** and optionally **Sub-Activities**.

3. Multi-Objective Template

- Multiple **Objectives**, each with their own **Activities** and possibly **Sub-Activities**.
- This represents a more complete plan or strategic initiative.

Tagging & Grouping OKRs [🔗](#)

- Templates can be **tagged with a Plan Name**, helping group OKRs under that plan.
- When OKRs are **assigned to a record (entity)**, the grouping follows the **Plan Tag**.
- On the **Plan Page**:
 - OKRs can be grouped **by Plan Tag**.
 - In the future, users may be able to **ungroup** them and **organize by priority** or custom lists, regardless of their plan affiliation.

When creating an OKR as a template, you'll be able to define which properties you want to use out of the available list.

The user isn't required to use all of them, except for Name and Measure unit. The rest can be left empty or tuned off for that ok - such as progress or traffic lights.

At this point, users can also choose to pre-fill some of the values within the template.

They might already want to define a target, or a due date, or attachments, or descriptions. That means that whenever that OKR is assigned to a record, it will be assigned with those values filled in as they came from the template. They can then be overwritten if necessary.

2. Key Metrics, Properties, and Behaviors [🔗](#)

Key Metrics represent a group of **Properties** that are related to business goals or measurable outcomes. These properties are not the same as **Entity Attributes**, as Key Metrics are used for aggregation and tracking specific performance indicators.

A **Component Metric** is a functional tool or feature within the platform that is built from a **Key Metric** – so, a component metric is a key metric that has been saved and defined. Component metrics use the properties of the **Key Metric** to deliver meaningful insights or track specific actions. Like for example, **OKR** (Objectives and Key Results) is a **component metric** built from a selected set of **Properties** from the **Key Metric**.

In Qollabi, component metrics are built using **Properties** from **Key Metrics**. Here's an overview of how these properties are applied to the standard component metrics of the system:

Properties of a Key Metric [🔗](#)

Property	OKR Component Metric	Guidance Component Metric	Survey Component Metric
Name	Default - Mandatory	Default - Mandatory	Default - Mandatory
Description	Default - Optional	Optional	Optional
Measure Unit	Default - Mandatory	Optional	Default - Optional
Progress	Default - Optional	Optional	Optional
Hierarchy	Default - Mandatory	Optional	Optional
Responsible	Default - Optional	Default - Optional	Optional
History	Default - Mandatory	Optional	Optional
Timeframe	Default - Optional	Default - Optional	Optional
Frequency (milestone)	Default - Mandatory	Optional	Optional
Attachment	Default - Optional	Default - Optional	Default - Optional
Due Date	Default - Optional	Default - Optional	Optional
Trend	Optional	Optional	Optional
Muted/Unmuted	Default - Mandatory	Optional	Optional
Shared/Private	Default - Mandatory	Default - Mandatory	Default - Mandatory
Comments	Default - Optional	Optional	Optional
Link	Optional	Default - Optional	Default - Optional

When we create an environment, the OKR component metric will have all the properties available to be used within that environment.

Later on, we might change this, e.g.:

- Customer A paid for **attachments**, but Customer B didn't. So, **attachments** won't be active in Customer B's environment.
- Different pricing groups might have different properties available.
- A customer might want to, on purpose, deactivate a certain property.

OKR templates